AI ENGINEER / AGENTS & AUTOMATION

Portfolio

“Turning repetitive field work into AI systems that run every day”

Jinhee Mok

AI Engineer · From design to deployment and operations

At StyleSeller I build and personally deploy and operate KakaoTalk/email record organization, vendor CS, internal work tools, and an agent platform. I don't call it done until I've run it on staff PCs myself.



Status board in production: stage bar, consolidated summary, and calendar for one counterpart (names blurred)

01
Status Board Operations Ledger
KakaoTalk, email, and Slack records
for hundreds of counterparts, one screen

02
Content Autopilot
From Reel request to approval
publishing defaults to manual after approval

03
Vendor CS Agent
Answers tagged questions only, no handoff
amounts and brands checked against code

04
Agent Platform · Spark
CEO routing, Jev skill recommendation
tool gates, cost caps

05
Knowledge Hub Email Agent
13 LightRAG workspaces
drafts only go out with evidence

06
Internal Work Tool
Pricing calculated by code
automated personalized KakaoTalk sends

07
Staff PC Deployment & Ops
One-line install across 11 PCs
per-device keys

Jinhee Mok
Jinheemok815@gmail.com

STYLESELLER
https://jin-tonix.github.io/

Portfolio

jinheemok815@gmail.com · +82 10-9030-2888
jin-tonix.github.io
Yeoksam-dong, Gangnam-gu, Seoul

Jinhee Mok

AI Engineer · From design to deployment and operations

Contact

Jinhee Mok

Skills

Current work (StyleSeller):

AI / Agent
Gemini Flash (via LiteLLM), Claude Code, 4 MCP servers, tool calling

RAG · LLM Ops
LightRAG, Neo4j, pgvector, LiteLLM

Backend
Python, FastAPI, TypeScript, Node.js

DB
PostgreSQL, PostgREST

Field PC · Infra
Chrome Extension MV3, Playwright, CDP, Swift, n8n, Docker, GitHub Actions, GCP, nginx

Training program (2024):
Java, Spring Boot, Vue.js, Flutter, LangChain, RAG, Tesseract, T5, NLLB

About me

"Turning repetitive field work into an AI system that runs every day"

I move work that people repeat by hand every day into AI agents and automation, then deploy and run the system myself.

I joined StyleSeller in November 2025 and work as a PL in the business division. Starting with seller DM automation, I've since built and now run Vendor CS, the internal work tool, the email agent and knowledge hub, Spark/Work OS, and staff PC deployment.

I hand much of the coding to Claude Code, but I decide what to build and check whether it actually works. I don't call something done until I've run it on an actual staff PC.

I studied law and worked as an English teacher and a daycare operations manager before moving into development. Automating receipt review with OCR and an LLM as an intern at We:Review was the start of the work I do now. I want to keep building workplaces where systems handle the repetition and people focus on judgment.

Journey

2025.11 - Present	**StyleSeller · PL (AI & Automation)** Designing and running internal AI/automation (2025.11~)
2025.08 - 2025.10	**We:Review (Welead) · Intern** Built an OCR + LLM receipt-review pipeline
2025.05 - 2025.07	**Korea IT Business Promotion Agency** Generative AI developer training program
2024.10 - 2024.12	**Himedia Academy** Ran an AI project under an industry mentor's guidance; collaboration and problem-solving
2024.04 - 2024.10	**Himedia Academy — Career Change** Full-stack development skills, collaboration and version control
2022.06 - 2024.02	Montekids — English teacher
2021.02 - 2023.06	Academic Credit Bank System — Business Administration
2021.03 - 2022.05	Pots&Pans — English teacher
2018.08 - 2020.09	JLS (Jeongsang Language School) — Teacher
2015.03 - 2018.02	Jeonwon Daycare Center — Operations Manager
2008.03 - 2012.02	Cheongju University — Law

At a Glance

Seven systems I built and run at StyleSeller, plus one follow-up improvement. Each case explains how I built it and why I decided it that way. Figures are stated as of each case's own reference date.

Status Board · Operations Ledger

Collects KakaoTalk, email, and Slack records into a per-counterpart calendar and shows deal stages for hundreds of counterparts on one screen

Content Autopilot

From Reel request through render and approval, all in the staff screen; publishing defaults to manual after approval

Vendor CS Agent

Answers only tagged questions in vendor group chats, no handoff, and checks amounts and brands against evidence in code

Agent Platform · Spark

Starts from a CEO work order, with the server controlling the dispatcher, workers, shared skills (Jev recommendation), and tool gates

Jev Skill Router Upgrade (Spark follow-up)

Made the Hermes agent's skill selection (Jev) accurate and fixed measurement errors. Confirmed that letting Jev filter first can also cut cost to about 1/10

LightRAG Knowledge Hub · Email Agent

Four agents find evidence from the same knowledge graph, and only evidence-backed drafts reach the owner's inbox

Internal Work Tool

Moved spreadsheet pricing formulas into code; the LLM only returns the cells to change. Automated personalized KakaoTalk sends

Staff PC Deployment & Operations

Deployed KakaoTalk and Instagram tasks to 11 staff PCs with a one-line install

Board

Status Board · Operations Ledger

StyleSeller · internal, private repository

Pick one counterpart and KakaoTalk, email, and order sheets land on one calendar, with deal stages for hundreds of counterparts on one screen

Period	Built 2026.10.19–10.23 (5 days), in production
Role	Design: ledger schema (a table that records each fact only once) and counterpart-linking rules / Build: intake API, mail collector, 6 screen tabs / Verification: DB tests, production screens. Used an AI coding tool, set the scope myself, and confirmed results through testing
Repo	About 150 status-board-related commits across 2 internal repos

Background

Conversations with counterparts are split across KakaoTalk rooms, the order mailbox, and Slack, and Excel order sheets arrive separately as email attachments. The status board summarizes these records by day and puts them in date order on each counterpart's calendar. Pick a brand, vendor, or seller, and every record exchanged with that counterpart shows up on one screen. The communication-status tab shows which stage each of hundreds of counterparts is at, from first contact to settlement. Records the rules can't link, or stages that are ambiguous, are left for staff to decide.

Structure and Scale

1. Tech

React 19.2 · Tailwind 4 · Vitest 4, Python 3.12 · Pydantic v2, Postgres RLS (a DB rule that limits who can see each row), Gemini Flash family (via LiteLLM)

2. Scale (dated per item)

- Communication-status stage mix: first contact 13.1%, proposal 33.8%, negotiation 2.6%, contract 4.5%, in progress 46.0%, settled 0% (2026-10-24)
- About 100–200 records per weekday, hundreds pending in the counterpart-check queue (2026-10-24)

3. How it works

① KakaoTalk comes in 3 times a day (09:15, 13:00, 17:55), the order mailbox every 5 minutes, and Slack through a general-purpose connector. However many times the same record arrives, the ledger keeps it only once.

② Newsletters are filtered out by rule before the LLM ever sees them, and in order-sheet tables, columns like the recipient name are found and the buyer's name is blurred. If a summary's cited phrase isn't in the original text, the summary is dropped.

③ No LLM is used to link counterparts — only rules. It checks, in order, mail thread, business registration number, KakaoTalk room, email, phone, domain, vendor code, and alias; if the keys disagree, the record stays unclassified.

④ In-progress deals group KakaoTalk contract signals and email signals by counterpart · target · round. A stage is applied automatically when the rule evidence is clear; the rest go to "awaiting review" for staff to decide.

⑤ Every day at 18:05, summaries for up to 500+ counterparts are rewritten into 3–6 lines. If the daily cost passes KRW 2,000, or the cost ledger can't be read, that day's batch stops.

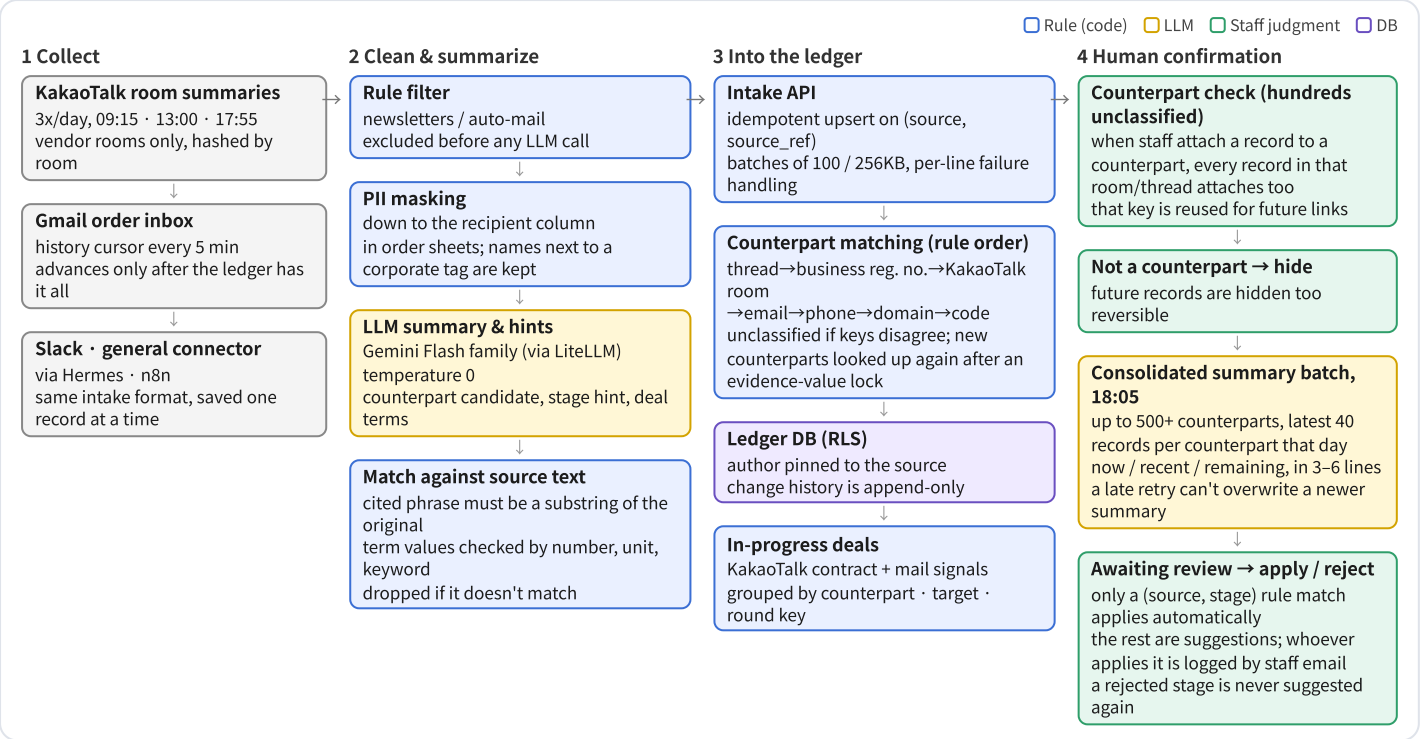
Incidents and Fixes

1. **All 107 summaries rejected (2026-10-23).** The DB function that saves summaries lacked execute permission for the app role, so all 107 summaries that day failed at the save step with a db_error. Fixed with a single GRANT line scoped to just that function.

2. **Over-eager "in progress" suggestions in rehearsal (2026-10-23).** The rule treated words like "shipping," "inventory," or "sales" alone as a deal signal. Changed it to keep only genuine deal-signal words and first strip negating phrases like "discontinued" or "order inquiry" before matching.

3. **A shared chatbot shown as the owner (2026-10-23).** The owner field for 64 counterparts showed a shared chatbot's name. The top 5 most active speakers in a KakaoTalk room were taken as the owner, and the chatbot was among them. Now only a company email account (@sample-corp.co.kr) is accepted as owner, falling back to the counterpart profile's owner if there isn't one.

Architecture



KakaoTalk, email, Slack → server (masking, LLM extraction, source matching) → idempotent intake (saved once no matter how many times it arrives) → work-tool ledger (RLS, rule matching) → staff confirmation

Screens


Screen A: per-counterpart calendar

Screen A: per-counterpart calendar. Pick one brand, vendor, or seller and KakaoTalk summaries, emails, and Excel order sheets land on one calendar in date order. Above it sit the stage bar, consolidated summary (now/recent/remaining), in-progress deals, and the counterpart wiki.


Screen B: communication status

Screen B: communication status. Shows the counterpart mix by stage (13% first contact to 46% in progress) and "awaiting review" suggestions that staff apply or reject.

Technical Highlights

1. The screen stays the same even when the same record arrives again. The collector resends the same record any number of times. It's saved with a single upsert statement that updates on a matching (source, source_ref). Rows staff have edited, and rows whose content hasn't changed, are left untouched. If the content is the same, the trigger doesn't bump the sequence or timestamp, so the screen stays unchanged after a resend.

2. The "author" value never comes from the caller. The intake function discards whatever author value a caller sends and rebuilds it as "ingest: <source>". A "manual" source is rejected, since that's reserved for staff. The "edited by staff" flag is stamped by a trigger, not the app, and a hidden record can't be revived with UPDATE. The app role only ever gets execute permission on functions.

3. Mismatched keys are never linked. The link order is fixed: thread, business registration number, KakaoTalk room, email, phone, domain, vendor code, alias. If a business reg. no. or phone number points to a different counterpart than other evidence, it's flagged as a conflict and left unclassified. 13 shared domains like gmail and naver are never used as evidence.

4. Concurrent collection never creates a counterpart twice. Only when creating a new counterpart does it take an advisory lock (a transaction-scoped name lock) per evidence value, in sorted order, then re-check; if another transaction already created the counterpart in the meantime, that one is used. Staff applying a stage follows the same lock order as collection (counterpart → suggestion), so there's no deadlock.

5. LLM output is checked against the source text. A stage's cited phrase is kept only if it's a substring of the masked body. Deal terms are held to a stricter check: every number in the value must appear in the quoted text, and the keywords and units must match the term type. A company name is used only when a normalized name for it appears right in the evidence line.

6. Order-sheet tables are masked column by column. Order sheets usually come as tables. It finds header columns like "recipient" or "orderer," and only masks that column's value on rows whose cell count matches the header row, so product names and quantities stay in the summary. A name next to a corporate marker like "(Co., Ltd.)" is treated as a company name and left unmasked.

7. Cost is reserved before the call, not after. The balance check and the reservation are one SQL statement, so two concurrent calls can't jointly exceed the cap. The reserved amount is whichever is larger: an estimate based on max input/output, or a minimum reservation. If settlement fails, later calls stop. A late retry's summary is blocked by a date comparison and can't overwrite a newer one.

8. The mail cursor only advances after the ledger has it all. Mail is queued locally first, then sent to the ledger. The Gmail history cursor only advances once that whole batch has succeeded. If the cursor expires, it refills the list from the last success time. Mail that fails twice is quarantined and raises an alert.

Status, Judgment, Limits

- In production. 6 tabs, KakaoTalk/email/Slack intake, and the 18:05 summary batch are running (confirmed on the production screen, 2026-10-24).
- Python, TS, and SQL all share the same name/phone normalization answer file. CI runs content-verification tests under the real app role on PGlite, and screen tests run outside CI.
- Limit: I haven't measured the rate at which staff applied "awaiting review" suggestions, i.e. suggestion accuracy. The server trusts the intake-side checks and doesn't re-verify cited phrases.
- If I did it again: I'd build a test for every new DB function that calls it as the app role. That test would have caught the 10-23 rejection of 107 summaries before it ever shipped.

What I Decided

- Decided to link counterparts by rule order instead of an LLM. If keys disagree, it's never auto-linked — it goes to staff.

Content Autopilot

StyleSeller · internal, private repository

Covers everything from a Reel request to a rendered 9:16 MP4 and approval in the staff screen. Live SNS posting is simulated

Period	2026.10.03–present (after the design doc was approved)
Role	Design: stage state machine and publishing defaults. Build: orchestrator, publish gate and DB slots, QA gate. Verification: production canary (a test run of one item first) and restart-scenario testing. Used an AI coding tool, set the scope myself, and confirmed results myself
Repo	Orchestrator, content agent, video renderer, work tool (content screen)

Background

A staffer requests a Reel from the work tool's content screen, reviews a preview, approves it, and schedules the posting time. Since generation costs money and publishing means external exposure, every default was set to "don't do it." Publishing is manual, recurring automation runs as a dry-run (goes through the motions without actually executing), and paid generation is off. Code, not a human, drives stage progression.

Structure and Scale

1. Tech

Paperclip sub-issues, orchestrator (a state machine with no LLM), Hermes and the Gemini Flash family (via LiteLLM), image-service, TTS, video renderer, HyperFrames (a tool that renders video from HTML), ffprobe, Google Drive, Blotato (SNS scheduled-posting service), Supabase (PostgreSQL)

2. Scale (dated per item)

- 3 services, 122K lines combined, 617 test files (orchestrator 135, Hermes agent 396, video renderer 86)
- Production canary verified end to end through saving a 1080×1920 MP4 to Drive (2026-10)
- 0 live SNS posts. One post per account per day enforced by the DB; the publish gate caps at 10 per day (2026-10-24)

3. How it works

① A staffer requests a Reel, or triggers a saved automation (immediate, scheduled once, or recurring).

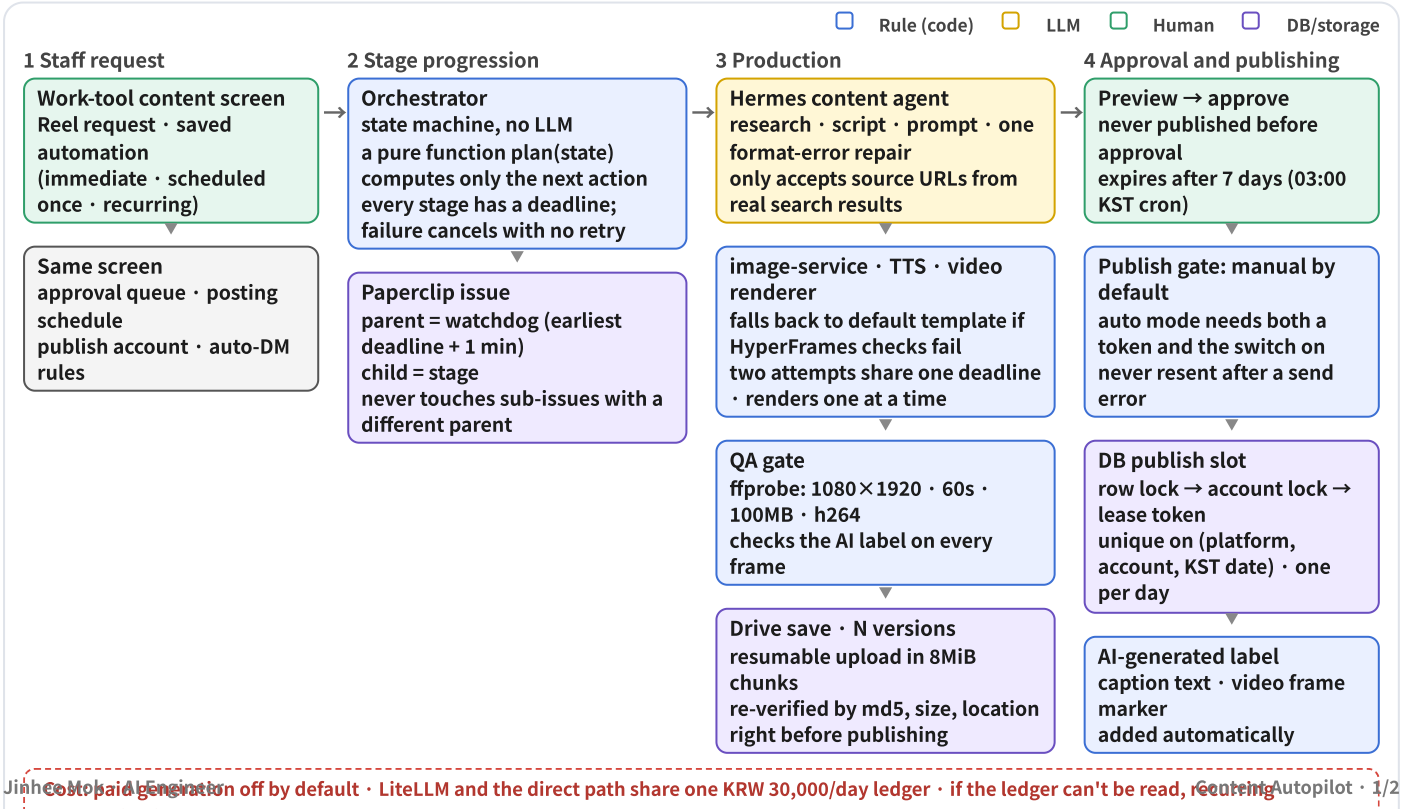
② The orchestrator's pure function computes only the next action; the executor applies it to a Paperclip sub-issue and advances the stage.

③ The Hermes agent writes the research, script, and prompts, and the video renderer renders a 9:16 MP4 from a HyperFrames template.

④ A version that passes the QA gate is saved to Drive with a checksum. A human approves it from a preview, and a pending approval expires after 7 days.

⑤ Publishing defaults to manual, so the person downloads the MP4, copies the caption, and clicks "published." The "AI-generated content" label and frame marker are added automatically.

Architecture



What's Blocked by Default

1. Publishing. Defaults to manual. Auto-publish needs both a token and the switch on. A partial unique index on (platform, account, KST date) allows only one post per account per day and rejects any more. A migration halts first if existing duplicates are found.

2. Paid generation and recurring automation. The paid backend defaults to False. Recurring automation defaults to dry-run and only unlocks when the value is exactly "false". It reads the company's KRW 30,000/day ledger before starting, and holds if it can't read it. The cap is bound with min() so it can't be raised via an env var.

3. Instagram auto-DM. The on-switch defaults to off, and dry-run defaults to on. Limited to 200/hour per account, and a malformed setting is read on the safe side. A (account, comment) unique index stops a DM going to the same comment twice.

Technical Highlights

1. A pure function decides the next action. plan(state) only returns a list of actions; the executor applies them. Every stage has a deadline, and the parent issue wakes at the earliest deadline + 1 minute to watch. On failure it cancels and closes the sub-issue with no retry. It never touches sub-issues with a different parent, so re-running never spins up a new pipeline.

2. A format error only gets one repair attempt. The repair target is JSON format errors only. It sends the error list (max 8 items, 120 chars each) without the prior output, and calls at most twice. A test pins down that a third failure after repair ends in failure, with no third call.

3. Sources are only accepted from real search results. A cited URL only passes if it's in the web tool's results or the input; an evidence field the model wrote itself is discarded. Merely claiming a search was done fails as web_research_unverified. Facts like product price are still blocked only by prompt rules for now.

4. A post is never resent when the outcome is unknown. Right before the POST, a start marker is written; if an error follows, it's set to publish_pending instead of failed. If it turns out to have already been published on restart, it closes as already_published. Blotato's submission ID is saved immediately, so waking up again only queries status.

5. The DB hands out publish slots. A claim function locks the row with FOR UPDATE, takes an account advisory lock, checks the daily cap, then issues a 30–900 second lease token (a temporary lock marker). Chose this over an in-process memory lock so the decision is made in one place in the DB no matter how many processes are running.

6. One ledger enforces two policies. To close the hole where the direct path bypassed the proxy's cap, LiteLLM and the direct Vertex path add cost to the same Redis key. If the ledger can't be read, an individual call still goes through, but starting recurring automation is held. Reads use a Redis user with no write permission.

7. Render retries share one deadline. If the motion plan the LLM wrote fails the HyperFrames check, it re-renders with the default template. The two attempts share one deadline, so a retry doesn't double the wait time. Rendering runs one at a time.

8. The file to publish is pinned by checksum. Right before publishing, it re-checks the Drive file's mime type, size, md5, and location. The QA gate uses ffprobe to confirm 60 seconds, 1080×1920, 100MB, h264, and checks the AI label on every frame of an AI video. Uploads read in 8MiB chunks and resume on a 308 response (the resume signal).

Status, Judgment, Limits

- This new production path is technically complete, but details like the prompts inside it are still being tuned. Content made through this path has no live SNS track record yet; the production canary has been verified through saving an MP4 to Drive (2026-10). Existing videos are already being published live, through a separate path that only swaps the caption and hashtags per account and posts to several accounts at once.

- Approval expiry runs on a daily 03:00 KST cron, in order: reject, cancel the issue, close the row. Even if canceling the issue fails, the row still closes, so the same failure doesn't repeat.

- Limits: product-fact verification is still prompt rules only, and registering a new Instagram token needs a developer to configure it. This path has no live posts yet, so I haven't measured outcomes like views.

- If I did it again: I'd add a code check that cross-references product facts like price and specs against the product DB. Right now, if the model breaks a rule, a human has to catch it in preview.

What I Decided

- Stage progression is owned by a state machine with no LLM; the LLM is only used for research, script, and prompts. Handing the next step to the model would make results non-reproducible across re-runs.

- Nothing auto-retries beyond one format repair and one default-template re-render; everything else closes. Anything that costs money or goes public needs a human to press it again.

- Publishing defaults to manual, and the daily cap lives in a DB index, not app config.

Vendor CS Agent

StyleSeller · internal, private repository

A KakaoTalk chatbot that answers only tagged questions, with no handoff, and corrects numbers and brand names against evidence

Period	2026.06–present
Role	Design: collection path, vendor gate, evidence-check rules / Build: server agent, send and delivery confirmation in the kakaocli fork (a CLI that reads and sends from the Mac KakaoTalk DB), Windows collector / Operations: incident response, the 09-02 response-style switch. Used an AI coding tool, set the scope myself, and confirmed results through testing and measurement
Repo	Server (vendor agent), kakaocli fork, CS Copilot extension · local sender

Background

Staff PCs collect conversations and upload them to the server, and replies are sent from staff Macs too. The server agent searches each vendor's records and the LightRAG knowledge hub (an entity/relation graph RAG with 13 workspaces) to write answers. Since 2026-09-02, it has stopped asking clarifying questions and answers only tagged questions, with no handoff. Instead, it checks amounts, percentages, and brand names against evidence, and if they don't match, searches again and rewrites.

Structure and Scale

1. Tech

macOS kakaocli fork (Swift), Windows read-only collector (Python), Python 3.12, Gemini Flash family (via LiteLLM), LightRAG 1.5.6, Google Sheets · Drive

2. Scale (dated per item)

- Vendor-agent code, tests, and config: 61K lines combined, 600 commits, 107 test files (2026-09-23)
- CS Copilot extension · local sender: 631 commits, 20,596 lines, 56 tests (2026-09-23)
- Tests: vendor gate 20, evidence judging 16, liveness markers 30, Mac Swift 34, Windows Python 72

3. How it works

① On Mac, kakaocli reads only what's after the last logId in the KakaoTalk DB; on Windows, the read-only collector uploads. It only moves on once the server answers 2xx.

② The server drops the same message if it arrives again within 15 seconds. It accumulates per-vendor ledgers, sheets, and Drive, then organizes them into a wiki twice a day (09:30, 18:10) and feeds the knowledge hub.

③ It checks the vendor ID or name in the room name against the work tool's list of vendors it serves. If the vendor isn't found on the list, it doesn't answer.

④ It only accepts questions tagged @company_AI. The regex only detects the tag; the model does the searching and writing. Other vendors' names are masked as [other vendor].

⑤ It checks the draft's amounts, percentages, and brand names against evidence, and if they don't match, searches again and rewrites. Replies are sent from a staff Mac, and delivery is confirmed by whether the KakaoTalk DB count went up.

Incidents and Fixes

1. A rule hijacked the decision (through 2026-08-30). The trigger regex was also doing the blocking, so some messages were silently blocked with no LLM judgment at all. Scaled the regex back to tag detection only and moved the answer-or-not decision to the model.

2. Sends got stuck (2026-08-17–18). On macOS 15, if the KakaoTalk window was in the background, the Accessibility (AX) call ignored its timeout and hung, leaving 31 of 147 sends (21%) permanently failed. Moved the call to a separate thread and had a semaphore cut off the wait past 3 seconds. Since 09-08, delivery is re-confirmed after every send.

3. A flood of past conversations (2026-09-16). An old build of the collector uploaded an entire conversation history, so 3 Windows PCs sent 2,500 messages in 50 minutes, the Sheets API got rate-limited with 429s, and the 2 staff who installed afterward had empty rows for over an hour. Had the server discard anything before a reference date and answer 200. Answering 5xx would have made the collector re-upload the same records.

Status, Judgment, Limits

- Collection, ledger/wiki organization, and tagged-question answering have been running since 2026.06. An answer a staffer sends directly in the room is learned right away, and the old answer to the same question is marked superseded.

- Vendor and influencer CS calls are excluded from the company's daily cost cap. The Windows collector has no auto-update, so a new build is deployed by reinstalling.

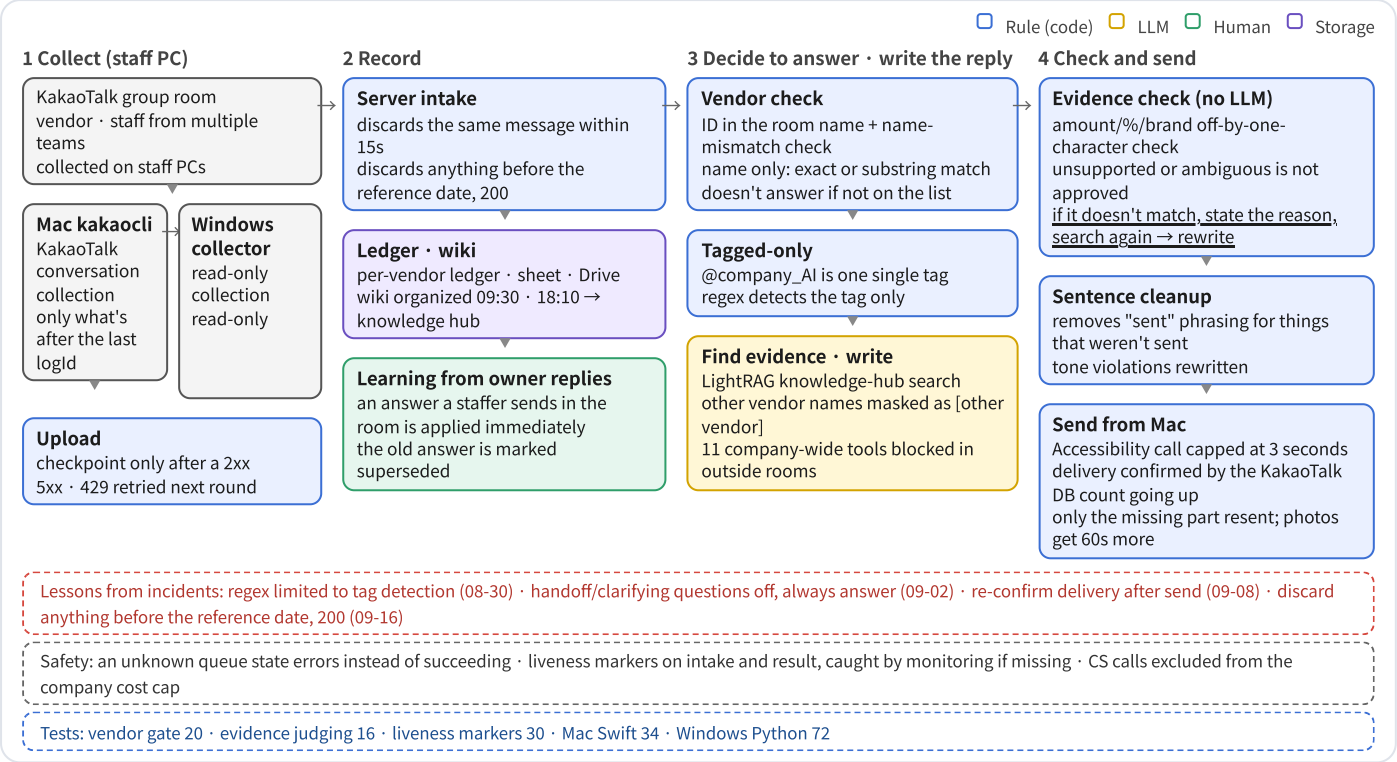
- If I did it again: I'd add auto-update and a reference date to the Windows collector from its very first build. The 09-16 flood of past conversations was the old build's fault, and reinstalling was the only way to fix it.

What I Decided

- On 2026-09-02, turned off clarifying questions and switched to answering only tagged questions with no handoff. In exchange, number and brand checks are done in code, not handed to the LLM. The old handoff branch is kept in the code, guarded by a test.

- Left the regex to tag detection only and handed the decision to the model. The setup where regex also did the blocking was removed on 2026-08-30.

Architecture



Staff PC collection → server ledger · wiki → knowledge hub → writes an answer to a tagged question → checks evidence · rewrites → sends from Mac · confirms delivery

Technical Highlights

1. A similar name never decides the vendor. The "(1234)" prefix in a room name is read as the ID. If the name part matches a different vendor with 0.80+ jamo similarity, it's treated as an ID typo and blocked. With no ID, only an exact or substring-matching name is accepted. Common suffixes alone scored 0.81–0.88, so fuzzy matching was dropped.

2. One message arrived 51 times. kakaocli resends the latest message in an unread room every 2 seconds. Counted it out: 51 times for 1 message, 170–200 per minute. Drops resends within 15 seconds keyed on (room, message, unread count, sent-by-me). The unread count is part of the key, so closing and reopening the room still gets through.

3. Numbers and brands are checked in code. Amounts and percentages in the draft are normalized ("30k won" becomes 30000 KRW). A value not in the search evidence, or a word that differs by one character from the evidence brand, is flagged as a violation. When one turns up, it states the reason, searches again, and rewrites.

4. "I sent it" is only written when it's true. If the send API wasn't called that turn, a completed-tense sentence like "I've sent you the quote" is removed. On a retry with the same idempotency key, it checks real queue state. In-flight or already-sent items aren't re-queued; only failed ones are retried. If the state is unknown, it errors instead of claiming success.

5. Other vendors' cases are masked. If a vendor name other than the current one shows up in a search result, it's swapped for [other vendor] and the model is told it was swapped. All 11 company-wide Spark tools are blocked in outside rooms. The vendor name used in a product recommendation lookup is set by the server, not the model.

6. Delivery is counted by the KakaoTalk DB count. A baseline count is taken before sending. After sending, it counts how many texts and photos appeared in the local DB and resends only what's missing. Photos get 60 more seconds of wait. In a 70-person room, the same photo going twice was judged less bad than one photo going missing. Reporting completion to the server also happens only after this check.

7. Checkpoints only move after a 2xx. The uploader only advances its offset after receiving a 2xx. 422 and 413 are skipped; a 5xx or 429 stops with no commit and retries next round.

8. The processing path leaves its own liveness markers. It stamps a marker once on accepting a tagged message and once on reaching a result. If no result marker shows up past a business-hours threshold, monitoring catches it. The old approach, where monitoring assumed "no draft means it's stuck" from outside, was dropped in review. This path has 30 tests.

Agent Platform · Spark

StyleSeller · Internal private repo

No matter who asks, the work starts from the CEO agent's work order, and the server controls delegation and tool execution

Period	2026.05 to present (kicked off 05-17, Spark/Work OS design 07-18~19)
Role	Design: work order, delegation guard, tool gate structure. Build: shared tool/skill box connection, one-time approval, LiteLLM cap, Jev router connection. Verify/operate: wrote the evaluation set and deployed to the VM. I used an AI coding tool, I set the scope, and I verified the results myself
Repo	Internal repo (Paperclip fork), skill folder, service code

Background

An internal platform where agents hand issues to each other to get work done. Forked from Paperclip (an open-source agent issue manager); company-specific features are attached as HTTP services under a service folder outside the core. The only exceptions are Work OS (the work-status/approval/execution-log app) and Spark. Staff use Spark to ask read-only questions against 4 business DBs and hand the actual work to the CEO agent. The longer a delegation chain runs, the more a request can drift, and any single tool can touch money or external exposure — so control lives in server code, not in prompts.

Structure and scale

1. Tech

Paperclip fork (Express 5, Drizzle, PostgreSQL), MCP SDK 1.29, Hermes (agent execution runtime), Python 3.12 with FastAPI, LiteLLM, Redis, Langfuse 4.50, Gemini Flash family (via LiteLLM)

2. Scale (as of 2026-10-24)

- About 5,000 of my own commits (excluding merges/upstream), about 2,200 test files (2026-10-24)
- 109 shared skills, 66 trigger evaluation files (1,282 sentences), 67 tool manifests

3. How it works

① Spark, Slack, and brand-tool requests all go to the CEO agent. Only structured brand-tool requests go straight to the content orchestrator, and a work-order record is still kept.

② The CEO writes a v1 work order. It includes the original text attachment with a sha256, the goal, completion criteria, constraints, and an approval flag.

③ The Hermes dispatcher under the CEO hands the issue to one of up to 10 department heads; if the switch is off, it falls back to direct CEO delegation. The server caps delegation depth at 2 and delegations per work order at 5, and blocks an A→B→A bounce-back.

④ BM25 narrows candidates from the 115 shared skills, Jev re-filters them by gate and fit score, and Gemini picks one skill from the result. If Jev fails or times out, the Jev stage is skipped and Gemini picks directly from the full list. The upgrade process is detailed on page 13.

⑤ Tools run through a 6-stage gate and human approval. Publishing is owned by the Publisher gate (a single service dedicated to publishing); the default is manual, and a DB index blocks more than one post per account per day.

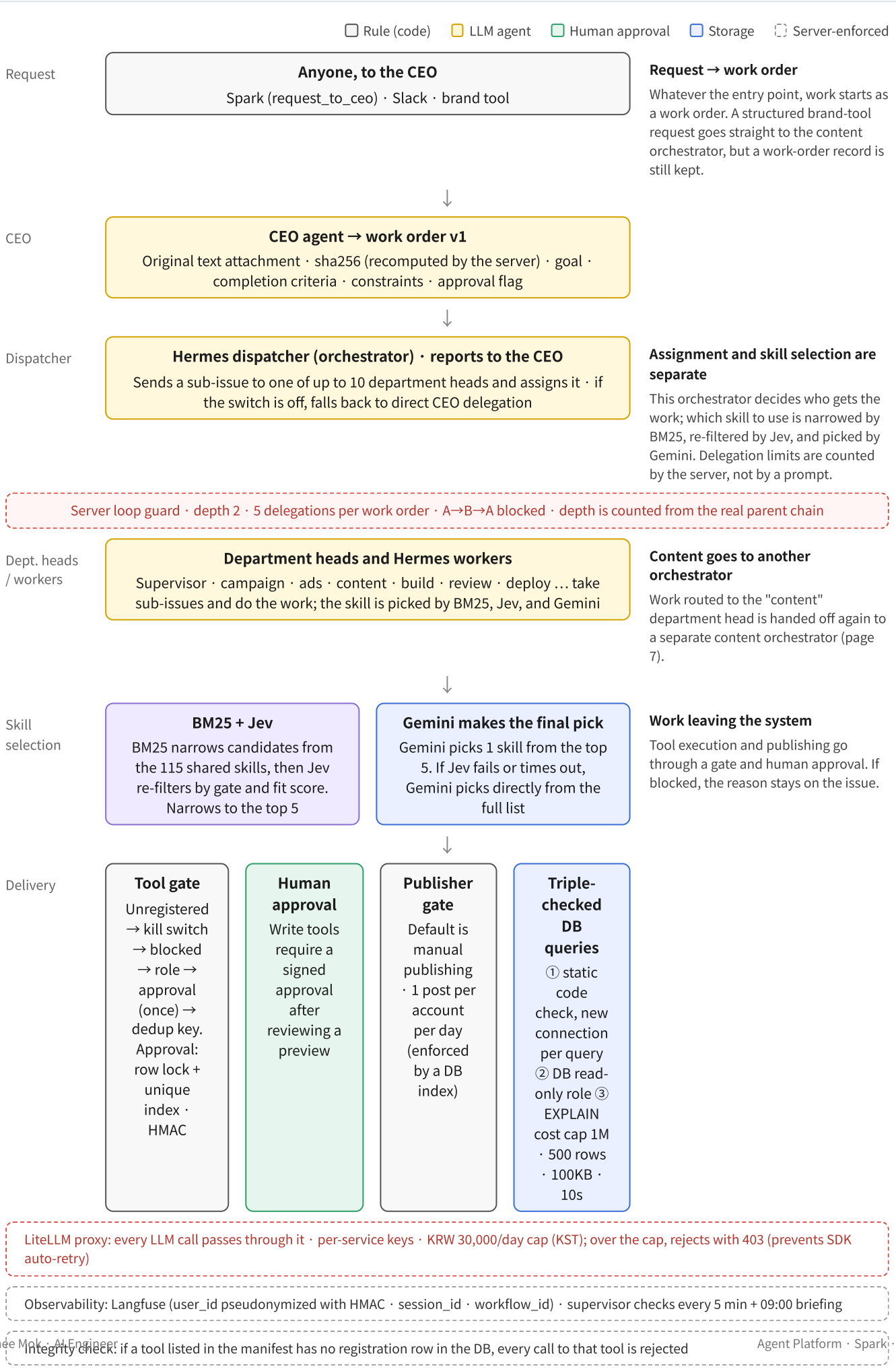
Incidents and fixes

1. Spark went down entirely (2026-10-24). A new Spark tool was missing its DB registration row, which failed the boot check and caused every Spark call to be rejected — the system behaved exactly as designed when a check fails. After recovering with a restart, I added a migration rule that inserts a registration row for every tool, plus a regression test.

2. A still-rendering job got reclaimed (2026-10-24). The server's reclaimer saw a video job that had run over 5 minutes, treated it as stalled, and reclaimed it; the result that came back 7 minutes 40 seconds later was rejected with a 409. Now a liveness signal goes out every 120 seconds but stops once the issue's deadline has passed, so a job that's truly stuck still gets reclaimed. In production I confirmed the signal was being received (200) and reclaims stayed at 0.

3. Made the Jev router lightweight (2026-10-23). For speed and to keep it light, I also moved the Jev router onto a lightweight model, behind an adapter that only matches the response format — the existing router code was left unchanged.

Architecture



Screen



Spark's answer screen. Question titles and counterpart names are blurred; each answer carries the name of the tool that provided the evidence

Technical highlights

1. The work order's original text is locked with a hash. The longer a chain of delegation runs, the more the request text can drift, so I NFC-normalize the original text, align line breaks, and lock it with a sha256. A hash sent by the client is overwritten by the server. I followed an external benchmark showing fewer supervisor errors when the original text is passed through instead of an intermediate summary.

2. The server cuts off delegation loops. A plugin can only watch after an issue already exists, so I fixed this in the server core instead. Depth is capped at 2, delegations at 5 per work order, and A→B→A is blocked. Depth is counted by walking up the real parent issue chain rather than a column a caller could tamper with. A 6th delegation and a forged depth are both rejected by tests.

3. BM25 narrows, Jev re-filters, and Gemini picks. BM25 narrows candidates from the 115 skills, Jev re-filters them by gate and fit score, and Gemini picks one. On failure, timeout, or a tripped circuit breaker (300 seconds after 2 failures), the Jev stage is skipped and Gemini picks directly from the full list. Rollout targets are split by the first 8 characters of a sha256.

4. Approval is spent exactly once. The approval row is locked with FOR UPDATE, and a partial unique index blocks it a second time. Approval is consumed even if execution fails. The dedup key uses ON CONFLICT DO NOTHING followed by a re-read, so there's no race. Tests confirmed 1 execution out of 10 concurrent requests.

5. A write approval signs the preview. The input hash, preview hash, requester, execution ID, and expiry are bundled into canonical JSON and signed with HMAC-SHA256, so even DB write access isn't enough to get through without the key. The dedup key has no timestamp in it, so retries don't rack up extra approvals.

6. Spark's DB queries open a fresh connection every time. A read-only role alone wasn't enough. One set_config call can undo a timeout and read-only setting, and a reused connection would carry that state to the next caller. After filtering 23 dangerous functions, semicolons, and write CTEs, it opens one connection and closes it when done.

Status, judgment calls, and limits

- In production. The company-wide switch for tool execution defaults to off, and each tool is turned on individually. User_id is sent to Langfuse as an HMAC pseudonym.
- Evaluation (as of the model used at measurement time): on 69 questions with known answers, Jev's candidate recall (the share of answers that appear among its candidates) is 85.5% at rank 1 and 91.3% in the top 3 (2026-10-24).
- If I did it again: I'd check the manifest against tool registration rows in CI. The 10-24 outage only surfaced at boot, but it could have been caught before the merge.

What I decided

- I chose a single full index over role-based skill bundles. On a 57-item selection set, both approaches scored the same accuracy, 0.947 (p=1.0). But the bundled approach missed skills that are only invoked by name. The cost difference over 66 items was KRW 27.
- The dispatcher handles assignment; skill selection is narrowed by BM25, re-filtered by Jev, and picked by Gemini. Even if Jev fails, Gemini still picks directly from the full list, so work doesn't stall. The pick is always made by Gemini, never by the worker.
- Controls like the loop guard, one-time approval, and the paid-call cap are all placed in the server and DB, so they hold even if the model tries to break them.

Follow-up · Jev skill router upgrade (next page)

- Separately from Spark's BM25 hint, I tuned a router that uses the same Jev rules and measured a final accuracy of 90.15% on 132 new questions — a different metric from the recall figure above.
- This approach was rolled into the Hermes agent, which is Spark's execution runtime.
- The real point wasn't accuracy itself — it was filtering out answer-key errors, fake skills, and inflated scores from the measurement.

Jev

Jev Skill Router Upgrade

Follow-up to the Spark case · 2026.10.25 ~ 10.30
StyleSeller · Internal private repo

Trying to raise accuracy, I found I had to fix how I was measuring it first

This is a follow-up to the Spark case. Jev is a judge model (TypeSafe) that returns probabilities instead of text, and the "Jev router" covers everything from candidate narrowing to judging. The sub-agents that Hermes, Spark's execution runtime, assigns work to use the Jev router to pick skills. The LLM-proposer approach I tuned this time was rolled into it (the earlier case's BM25 narrowing was as of 10-24). The skill list grew from 109 on 10-24 to 115 by the time of this test. I designed the test, ran the experiments, checked the results, and made the rollout call; I used an AI coding tool to implement it, but I set the scope and verified the results myself.

~90%
Accuracy 90.15% (119 of 132; 1 case = 0.76pp)

The number changed four times
Answer-key errors, inflated scores, a fake skill, re-measurement

Rolled out
Fully applied; 3/3 correct on real production questions

Request → BM25+Jev: narrow & verify (gate → fits) → Gemini: final pick → 1 skill, or "none"

At first, the judge model alone with no BM25 topped out at 45-53%; adding an LLM proposer on 10-26 got it into the 80s. From there, 10-27 through 10-30 I pushed for 90%. The earlier case's 91.3% is BM25 candidate recall, measured on a different test set, so it isn't comparable.

Corrections and re-measurement

The answer key was wrong 10-27
Of 6 cases flagged as "the router called a tool for a question that didn't need one," 5 were the router being right and the answer key being wrong. I would have misjudged this if I hadn't opened each case myself. I kept the numbers from both before and after the fix.

The setting I picked using the test set looked inflated 10-27 · 10-29
On contaminated data, the test set I'd already seen gained +4.4pp and new questions gained +1.2pp; after cleaning up and re-measuring, that became 0 and +1.5pp. Either way the gain was small and unstable, so I stopped using scores from the test set I'd already seen as evidence.

A fake skill had gotten mixed into the test folder mixed in from 10-25 · found evening of 10-27
The same wrong answer showed up identically on two test sets, and I called it a "real bug" twice. Checking the 115 skills in the production repo one by one against the test folder, I found a 116th skill that existed only in the test folder. That's an error that couldn't happen in production, so I withdrew the conclusion.

After fixing it and rerunning, the numbers shifted again 10-29
A cross-review pointed out that "the comparison baseline was also measured on contaminated data," so I reran it from scratch. I cleaned the fake skill out of all 3 places and reran all 222 cases (90 + 132). The comparison setting I did not turn on would pick a skill with a fit score as low as 0.22 whenever confidence was high, and get it wrong. The setting I did turn on for production never did that — which is why I left the comparison setting off.

What was measured	Production setting	Comparison setting (not enabled)
① First measurement · 60 cases with known answers (answer-key errors still present, 10-27)	83.3%	91.7%
② After fixing the answer key · all 90 questions (10-27)	88.9%	93.3%
③ Re-measured after removing the fake skill · the same 90-question test set (10-29)	94.4%	94.4%
④ Same re-measurement · 132 new questions, reused afterward (10-29)	**90.15%**	91.7%

This is what went into production. BM25 narrows the candidates, Jev re-filters them with a gate and fits (per-candidate fit score) down to the top 5, and Gemini makes the final pick from those. The comparison setting turns on 2 more switches; its gain was only +1.5pp, and it carried a risk of answering wrong by routing around skills with a low fit score — so it was not enabled.

Production rollout and the cost experiment

- **Letting BM25+Jev filter first cuts Gemini's cost by about 10x.** I confirmed in a small test that BM25 narrows candidates from the 115 skills and Jev re-filters them by gate and fit score, then handing only the top 5 to Gemini, and then reran it on the same 132 new questions used for the LLM-proposer measurement. Gemini now reads 5 skills instead of 115, cutting cost by about 10x, while accuracy — 119 of 132, 90.15% — was about the same as Gemini reading all 115. Jev returns probabilities instead of generating text, so it's faster and cheaper than Gemini (TypeSafe's published figures are 70-500ms; my own measured median was 360ms). This two-stage pattern — a cheap model narrows candidates, then a stronger one makes the final pick — is in the same direction as the published SkillRouter paper (arXiv 2603.22455). This approach is now fully rolled out, and it's what the sub-agents Hermes assigns work to use to pick a skill today. If Jev fails or times out, Gemini picks directly from the full list. Basis and scope: 115 skills, measured on 132 previously-unseen questions; the 10x cost figure is an estimate based on public pricing for the number of skills Gemini reads.
- **All of this ran inside the rate-limit quota.** I compared settings from saved answers with no extra calls, scaling up from 20 to 90 to 132 questions. Across every test run, the rate-limit count (429) stayed at 0.

What I turned on · what I decided

I checked the 3 places the code changed to confirm they matched, backed up the existing config file, and added one line to it. I restarted the server, confirmed it came back up healthy,

What I didn't turn on · limits

I also tested a comparison setting that turns on 2 more switches, but didn't ship it. The accuracy gain was only about 1.5 percentage points, and when the model was confident enough,

LightRAG Knowledge Hub · Email Agent

StyleSeller · Internal private repo

4 agents find evidence in one shared knowledge hub; email only gets a draft with evidence into staff's inbox

Period	2026.03.31–present
Role	Design: workspace registry, search planning and evidence-judging steps, draft verification order / Build: email-agent server, checker, sent-mail learning cursor / Operate: draft-folder deployment, incident analysis and recovery
Repo	email-agent

Background

Vendor CS, Influencer CS, Spark, and sales-partner matching all share one LightRAG knowledge hub (13 workspaces). It's a RAG store that finds material for each question before answering. Every workspace carries a description, owner, channel, and sensitivity level, and using an unknown workspace fails immediately. For shared-mailbox inquiries, it puts a reply draft with evidence into the assigned person's Gmail drafts folder. Staff send it.

Structure and scale

1. Tech

Python 3.12 · FastAPI · Pydantic v2, LightRAG 1.5.6 (Postgres/pgvector + Neo4j 5), RAG-Anything/MinerU, FlashRank (lightweight reranker), Gemini Flash family (via LiteLLM), Gmail, pytest

2. Scale (as of 2026-10-24)

- About 1,470 commits · about 47,000 lines of product code · about 33,600 lines of tests (10-24)
- 27 routers · about 1,750 test functions · 13 workspaces (10-24)
- 18 search evaluation cases (query · expected workspace · required evidence · forbidden claims)

3. How it works

① An LLM planner (temperature 0) looks at workspace descriptions and the data catalog and builds up to 12 queries. Plans that fail validation are dropped, and the rule-based router takes over.

② The search mode (mix, combining vector and graph search) is fixed in code at each call site. Agent search uses top_k (results to fetch) of 8 and caches the same query for 60 seconds.

③ Evidence is judged by rule. Real-time values like price, stock, and settlement are forbidden claims, and refund confirmations, contracts, and bulk sends get a human-approval marker. Only the missing evidence items get a second lookup.

④ Rerank scores (a step that re-orders results) group results by topic. Within a group, official corrections and the latest documents go first, and results are interleaved one per group on output.

⑤ Up to 6 claims in a mail draft are checked against evidence by a checker and an LLM verifier. Claims without evidence are rewritten, and if unsupported claims still remain, the draft is dropped.

Incidents and fixes

1. Missing rerank step (2026-06-19 to 07-23). Starting the day after the 06-18 switch to agentic RAG (where the LLM sets the search plan), CEO mail-draft quality dropped. Going back through the commit history showed the rerank step had been dropped during the switch; restored it on 07-23.

2. Search overload (07-10, 08-12). The Influencer CS bot held the whole shared connection pool, pushing even matching search to 500s (07-10). A top_k 40 search triggered Neo4j overload warnings (analyzed 08-12). Split the connection pool per workspace and capped agent search at top_k 8.

3. Silent graph outage (2026-10-04). The container looked healthy even while Neo4j was down. Now /readyz sends Neo4j a RETURN 1 with a 2.5-second timeout and reports ok/fail, but this failure alone doesn't trigger a 503, so restarts don't loop. Search replies with 503 and Retry-After 30 after 2 attempts.

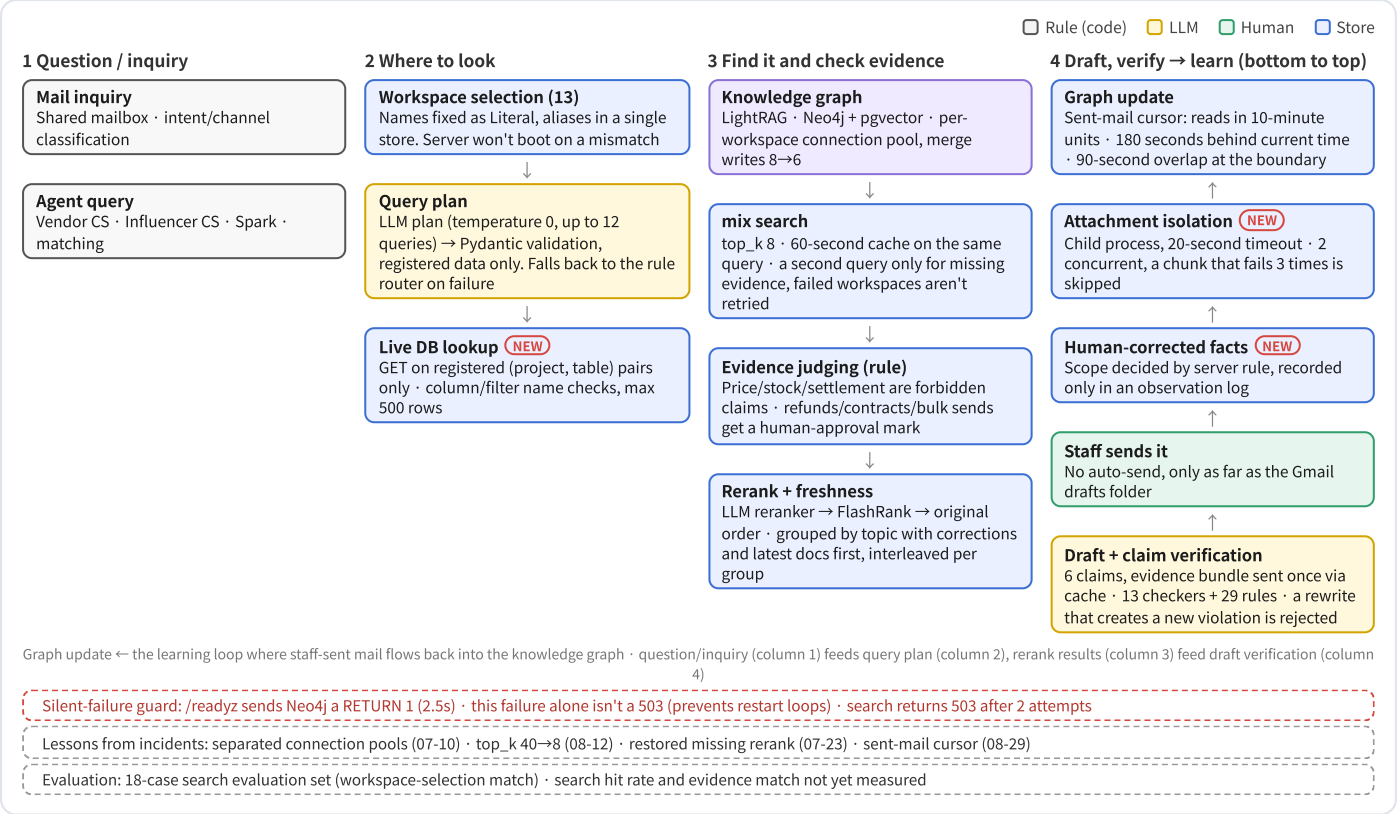
Status, judgment, limits

- Reply drafts go only as far as the assigned person's Gmail drafts folder. Sending is on staff, and deployment has used only images that passed pytest since 2026-08-18.
- Limit: the 18 search evaluation cases only check workspace selection. Search hit rate, evidence match, and the effect of restoring rerank are not measured.
- If I did it again: I'd add evidence-match scoring to the evaluation set first. That would have caught the missing rerank step before draft quality dropped.

What I decided

- The LLM planner decides where to search; code decides the search mode. I didn't let the agent choose the mode too.
- The only source of truth is mail staff actually sent. AI drafts are never fed back into the knowledge hub.
- Facts staff corrected aren't used right away — they go into an observation log first, to confirm the scope of the change.

Architecture



Sent-mail facts · KakaoTalk wiki · PDFs → knowledge hub (entities · relations) → LLM planner → search · evidence judging → draft · reply

Technical highlights

1. The server won't boot if a workspace name is off. If agents called it by different names, writes could land in the wrong store, so the 13 canonical names are fixed as a Literal type and about 50 aliases are collapsed into a single store. On import, a missing entry or store mismatch stops startup with a RuntimeError.

2. An LLM search plan is only used after it passes validation. Plans are checked with Pydantic. Live DB lookups only allow registered (project, table) combinations, GET only, max 500 rows. If the call, parsing, or allowlist check fails, it falls back to the rule router and the reason is recorded as fallback:<exception name>.

3. If rerank fails, two more steps stand in. Documents are cut to 20 and the LLM reranker runs first. If that fails, FlashRank; if that also fails, the original order gets scored so a result always comes out. Concurrent search steps were lowered to 3, which cut concurrent Neo4j writes in the merge step from 8 to 6 per document.

4. 13 draft checkers each own one error pattern. Patterns like a reversed request direction, mentioning a prior mail that never existed, re-asking a settled condition, or stating a proposed price as if it were final — each checker has an actual mail reproduction case from 08-21 to 08-23. It started as regex, but false positives kept coming, so judging moved to an LLM checker.

5. A fix that creates a new lie is rejected. Comparing only the sum of violation scores lets a 2-point factual violation pass as the price of fixing a 10-point banned phrase. If a rewritten draft introduces a factual violation that wasn't there before, that rewrite is dropped even if it fixed something else. A draft that still has violations after rewriting is logged per violation type as "Draft rejected" and discarded.

6. The evidence bundle is sent only once. Verifying 6 claims used to resend an evidence bundle of up to 150,000 characters each time. Now the bundle is cached for 5 minutes under a sha256 key and reused. If the cache fails, it falls back to the original method, so the judgment result doesn't change.

7. The sent-mail cursor accounts for Gmail's indexing delay. At deployment, 48 of 1,943 sent emails were left out of learning. Now it reads in 10-minute units and saves the cursor each unit. Since the search index lags, the cursor sits 180 seconds behind current time, and the boundary is read with a 90-second overlap. The cursor isn't advanced for a unit with more than 500 items.

8. The server decides the scope of a staff-corrected fact. Server rules judge whether a fact applies company-wide or to a single deal, so a single deal's fact isn't broadened to an entire partner. Results go only into an append-only observation log, run in shadow mode — never used in drafts or search.

Internal Work Tool · Pricing, Matching, Personalized KakaoTalk

StyleSeller · Internal private repo

Code calculates the amount; the LLM only returns candidates and the field to fix

Period	late 2026.03–present
Role	Design: price-calculation structure, rules for merging matching candidates, send-approval criteria / Build: pricing code, catalog extraction, matching, send automation, staff PC workers / Verify and operate: spreadsheet cross-checks, send/worker operations (seller-recommendation logic is owned by a colleague's CRM)
Repo	work tool, server (staff PC workers)

Background

Price calculation lived in Google Sheets formulas, and every quote was matched by hand. While moving that formula into code, I added a test that checks it field by field against real products' sheet values. For catalog product extraction and sales-partner matching, the LLM only produces candidates, and registration and review stay with staff. The seller-level recommendation for personalized KakaoTalk is decided by a colleague's CRM; I built the automation that receives those targets and sends.

Structure and scale

1. Tech

React 19.2 · Tailwind 4 · Vitest 4, Postgres+PostgREST (a REST server that opens tables directly), Gemini Flash family (via LiteLLM), knowledge-hub RAG search (finds material and attaches it to the answer), kakaocli, staff PC workers

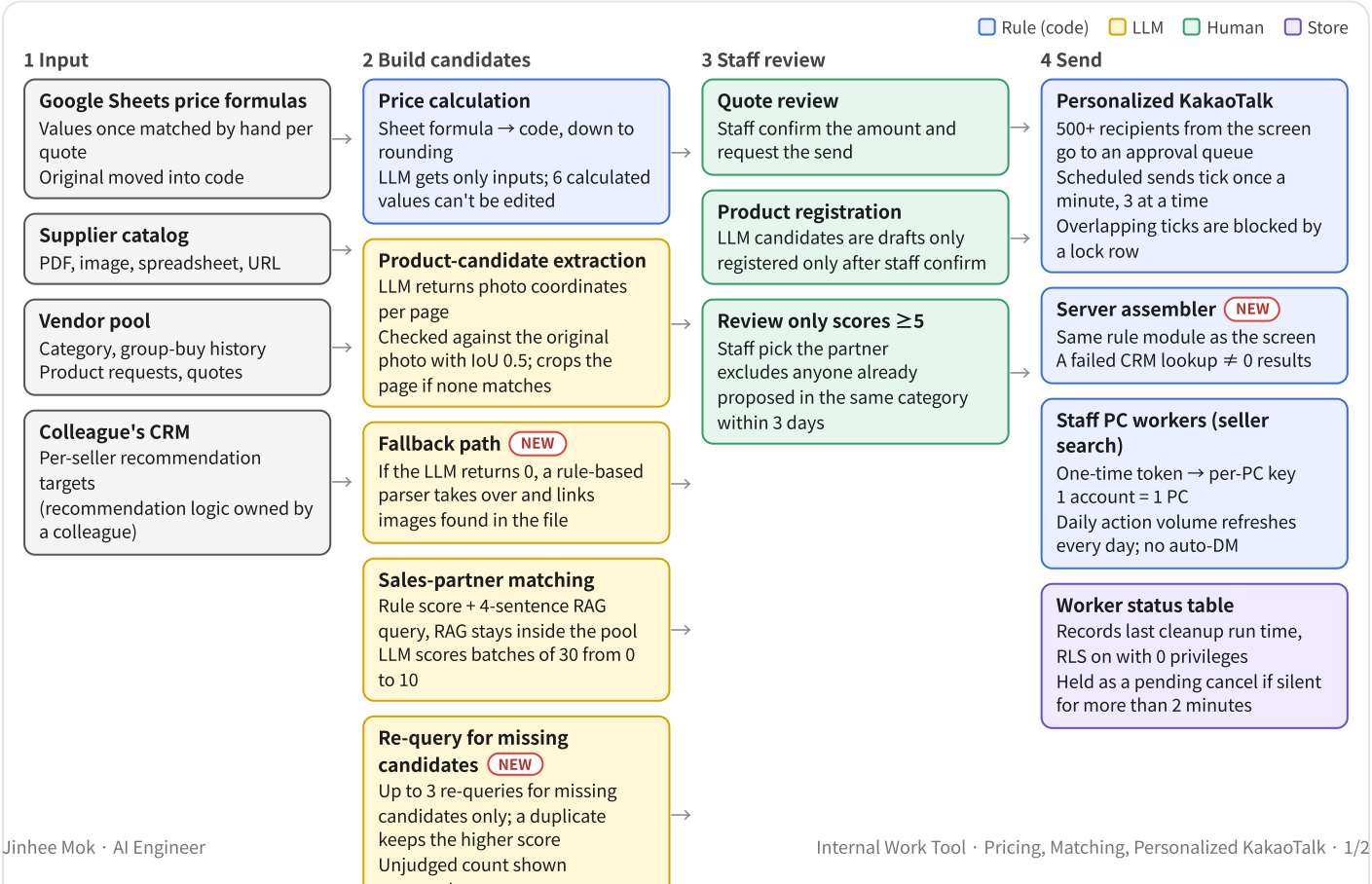
2. Scale (dated per item)

- Personalized KakaoTalk sent to 261 people in one day (2026-09-22)
- 43 price-calculation test cases, some cross-checked field by field against real products' sheet values
- Seller-search workers run on 9 staff PCs (2026-09-18)

3. How it works

① Pricing: Moved sheet formulas into code, down to the rounding unit. Even on an edit request, the LLM only returns the input values to change as JSON, and code recalculates the amount.
② Catalog: The LLM pulls product candidates from PDFs, images, spreadsheets, and URLs. Only items staff confirm get registered.
③ Matching: Rule-score candidates and RAG candidates are merged, and the LLM scores batches of 30 from 0 to 10. Only scores of 5 or higher reach staff, and anything already proposed in the same category within 3 days is excluded.
④ Personalized KakaoTalk: Staff send, schedule, and stop from a screen. 500 or more recipients go to an approval queue, and scheduled sends go out from the server a few at a time every minute.
⑤ Seller search: Staff PC workers run one PC per account, within a daily action cap. No auto-DM code was added.

Architecture



Incidents and fixes

1. Missing matching candidates (2026-07-15). In 3 of 6 chunks, the LLM ended its response before judging every candidate. From 07-16, candidates are merged into a union first and then asked about in batches; from 08-07, up to 3 re-asks target only the missing candidates. The count still unjudged shows separately on screen.

2. Shared worker key (2026-09-10). Every staff PC worker shared a single key. Switched to issuing a per-PC key from a 30-minute, one-time install token. The server keeps only the key's sha256, compared with timingSafeEqual.

3. 6 rounds cancelled at once (2026-09-20). When the server came back after a two-day stop, the "worker unseen for 3 minutes" rule read the server gap as workers dropping out and cancelled 6 in-progress rounds at once. Now, if the cleanup interval exceeds 2 minutes, it's treated as a server gap and the reference time is rolled back.

Technical highlights

1. I decided which fields the LLM can edit. If the LLM backed out a request like "set the seller's payout to 1,000," other amounts would shift too. So the 6 calculated values, like supply price and payout, are blocked from the edit list. For such requests, it only tells the user which input value to change — asking to change only the margin rate won't return a new sale price.

2. Moved the sheet formulas over down to the rounding unit. Partner supply price rounds up; the fee rate is fixed in code. Added a test that checks 2 real products from the sheet (e.g. a sample item at KRW 30,000) field by field. Also checks that total profit stays the same when the split ratio changes, and that the two shares always sum to total profit.

3. Looks for the original photo inside the PDF first. Sends each page to the LLM one at a time to get photo coordinates. If those coordinates overlap an original image embedded in the PDF by more than 0.5 IoU (the overlap ratio of two boxes), it uses the original. Backgrounds covering more than 80% of the page are excluded as candidates, and the page is only cropped when no matching original exists. A product spread across multiple pages has its empty price fields filled in from each other.

4. Excel still produces a result even if the LLM fails. If LLM parsing returns 0 items, a rule-based parser re-reads the same file instead of showing a blank screen. Images embedded in the file are attached to products in order. Icons under 750 bytes are dropped rather than treated as product photos.

5. RAG can only add candidates, never create new ones. Rule scores are ranked by category, group-buy history, product requests, and quotes, in that order, and 4-sentence RAG query results are added as a union. An ID RAG returns is only accepted if it's already inside the existing vendor pool. This logic used to live in n8n; moved it into code so it could be tested with vitest.

6. Automated sends and screen sends follow the same rule. The server assembler calls the same product-selection and template module as the screen, so the same person never gets a different product. A failed CRM lookup and 0 results are reported separately, and only 0 results get filled with popular products. The new-seller card is created only once — on 09-22, when a card was made per person, the renderer threw 980 instances of error 500.

7. Scheduled sends go out a little at a time each minute, without overlap. Targets with a personalized recommendation are assembled 3 per tick; targets without one, up to 30. If the unsent backlog reaches 5 or more, that tick is skipped. Once the send window has passed, it doesn't restart, so sends don't pile up overnight. A lock row per schedule prevents ticks from overlapping, and release requires the token left when the row was claimed.

8. One account runs on only one PC. Checking and consuming the install token are combined into a single conditional UPDATE, so two PCs can't register with the same token at once. If the same Instagram account is claimed by two PCs, the one that logged in more recently wins. The server's work-assignment function follows the same rule. Daily action volume is redrawn within range every morning.

What I decided

- I didn't leave amounts to the LLM. Code moved from the sheet does the calculation; the LLM only returns the input value to change.
- RAG search is only used to widen candidates. Vendors outside the existing pool can't get in.
- Sends of 500 or more don't go out immediately — they wait in an approval queue.

PC Deploy

Staff PC Deployment · Operations

StyleSeller · Internal private repo

Deploys and operates KakaoTalk and Instagram work with a one-line install on 11 staff PCs (as registered on 2026-09-16)

Period	2026.02–present (Instagram DM extension from 02, KakaoTalk tooling from 05)
Role	Design: install, authentication, and update structure, plus a server-side reference date (the cutoff before which messages are dropped) / Build: installer, workers, Windows collector, changes to the kakaocli fork / Operate: deployment, incident response, monitoring cron. Used an AI coding tool; I set the scope and checked the results myself
Repo	internal repo, kakaocli fork

Background

KakaoTalk and Instagram have no official API the server can call, so the work had to run on staff Macs and Windows PCs. Install is a one-line command, authentication uses a per-device key, and updates run on a tool-specific automatic cycle. I run the server myself too. On 2026-08-20, when a cloud DB limit stopped all work-tool APIs, I stood up the same API on a VM and recovered the same day.

Structure and scale

1. Tech

macOS bash and Windows PowerShell installers, launchd and Task Scheduler, Node workers, Python collector (Windows), kakaocli fork (Swift), VM Postgres+PostgREST+nginx, SSE (server-pushed events) push hub, Docker deploy scripts, Slack monitoring cron

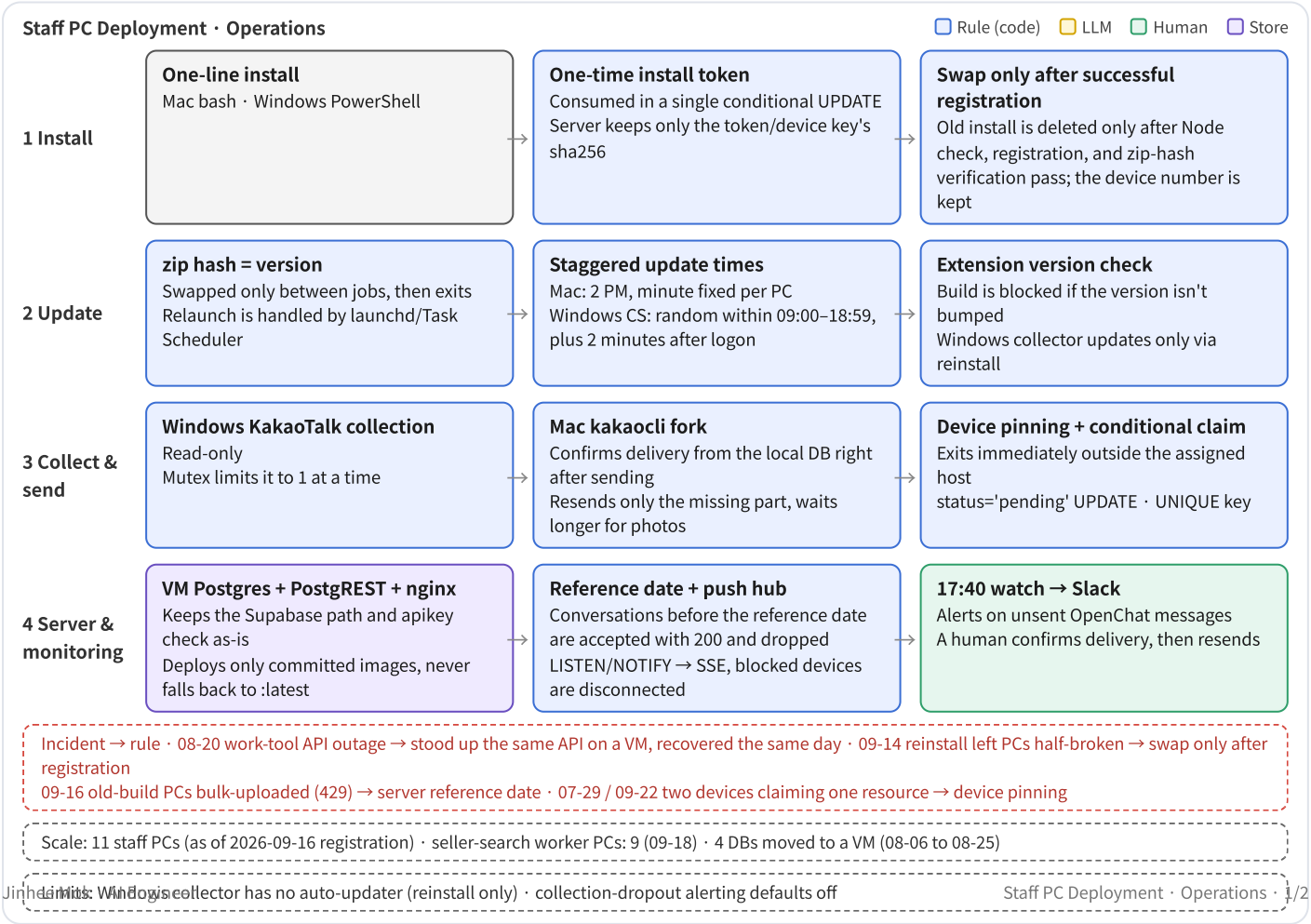
2. Scale (dated per item)

- 11 staff PCs (as of 2026-09-16 registration), seller-search worker PCs: 9 (09-18)
- Moved 4 DBs to self-hosting (2026-08-06 to 08-25)
- Recovered a full work-tool API outage the same day (2026-08-20)

3. How it works

① Staff install with a one-line command — bash on Mac, PowerShell on Windows.
② Registering with a one-time install token issues a per-device key. The server keeps only the token and the key's sha256.
③ Each tool has its own auto-update cycle, and update times are staggered per PC. Only the Windows collector updates via reinstall.
④ Queue jobs are only picked up by the assigned device. Claiming is a single conditional UPDATE, so two devices can never grab the same job.
⑤ The server deploys only from committed images. A 17:40 watch alerts Slack on unsent OpenChat messages, and a human resends after confirming.

Architecture



Staff PC Deployment · Operations

Incidents and fixes

1. Full work-tool API outage (2026-08-20). When a cloud DB transfer limit stopped all work-tool APIs, a VM's nginx was set up to mimic the Supabase path and apikey header check exactly. The app only changed a connection setting and a few adapters, left about 100 createClient call sites in the app code untouched, and recovered the same day.

2. Reinstall broke working PCs (2026-09-14). The installer deleted the old install before registering the new one. Running on an expired copy left a PC with neither config nor package. Now the old folder is deleted only after Node check, registration, and zip-hash verification all pass. The device number is kept across reinstalls.

3. An old-build PC uploaded its entire conversation history at once (2026-09-16). 3 Windows PCs sent their accumulated backlog all at once. The Sheets API got rate-limited (429), leaving the 2 PCs installed afterward with rows empty for over an hour. Fixing the client doesn't help until the PC gets the new build, so a reference date now lives on the server — messages before it are accepted with 200 and dropped.

Technical highlights

1. The install token is consumed in a single UPDATE. Separating the lookup from the consume step would let two PCs register with one token at once. An UPDATE conditioned on used_at IS NULL and the expiry only passes when it returns a row. Failure reasons are split into staff-readable messages. If one device is compromised, only that row is discarded.

2. Uses a tip hash instead of a version number. The published zip's sha256 is the version. With no separate version ledger, the ledger and the file can never drift apart. The worker swaps only between jobs and simply exits; relaunching is left to launchd and Task Scheduler. If it relaunched itself and two copies ended up running, the Instagram profile would break.

3. Update times are staggered per PC. If every device updated in the same minute, there'd be no window to pull back a bad deploy. The Mac real-time sync picks a random minute at 2 PM and keeps it across reinstalls. The Windows CS tool fetches at a random time between 09:00 and 18:59, plus a retry 2 minutes after logon.

4. Windows KakaoTalk uses a read-only collector. The collector only opens read access and never writes.

5. Send success is confirmed by the DB, not the exit code. On 2026-09-07, a send left out the text while the photo went through, and it was still marked sent. Now it reads the local KakaoTalk DB right after sending and immediately backfills only the missing part. If a photo doesn't show up, it waits longer to allow for a slow upload — in a 70-person room, sending a duplicate photo is worse than missing one.

6. Device pinning and conditional claiming overlap for safety. On 2026-07-29, a dev Mac picked up the same queue as a remote-only Mac. Successes and failures alternated, and it took a day to find the cause. Now anything outside the assigned host exits immediately. A claim is a conditional UPDATE on status='pending', and the OpenChat queue is further guarded by UNIQUE(send_date, room_name).

7. Device blocks are delivered immediately via push. The push hub takes Postgres LISTEN/NOTIFY and wakes workers over SSE. If device_keys changes, it re-authenticates before waking. A blocked device gets a revoked signal and the connection is cut — the server side only needs to fill in one revoked_at field.

8. deploy.sh re-runs using the version of the commit being deployed. The 2026-09-28 deploy failure happened because the running script and the committed script differed. The image tag is fixed once as sha-<commit>, and if the pull fails, it stops instead of falling back to :latest. Compose is rendered before the build so a missing variable is caught before a 40-minute build, not after.

Status, judgment, limits

- In production. The 11 staff PCs and 9 seller-search worker PCs are counts as of the 2026-09-16/09-18 registrations; counts since then aren't recorded here.
- The most frequent incident was two devices claiming one resource (07-29, 09-22), which is why device pinning is the default.
- Limit: the Windows collector has no auto-updater, so it updates via reinstall. Collection-dropout alerting is off by default.
- If I did it again: I'd add a zip-hash updater to the Windows collector from day one. The 09-16 incident grew because old-build PCs never received the client fix.

What I decided

- If there's no CI image, deployment stops instead of falling back to :latest, so a deployed image always matches a commit.
- Standing up the same API on a VM first, rather than fixing the app during the DB outage, meant the app-side change was just a connection setting and a few adapters.
- The reference-date filter never drops a message whose time it couldn't read. Having a little extra data show up is easier to see and fix than data quietly disappearing.